



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



Estructura de datos con memoria dinámica

Las estructuras de datos con memoria dinámica se usan para **optimizar** el uso de **memoria** cuando **almacenamos** datos

Estas estructuras siguen siendo **colecciones** de datos, pero con formas de acceso **diferentes**

Las estructuras de datos de memoria dinámica son:

- Listas enlazadas simples
- Listas enlazadas dobles
- Listas circulares
- *Pilas*
- Colas
- Árboles

PILAS (stack)

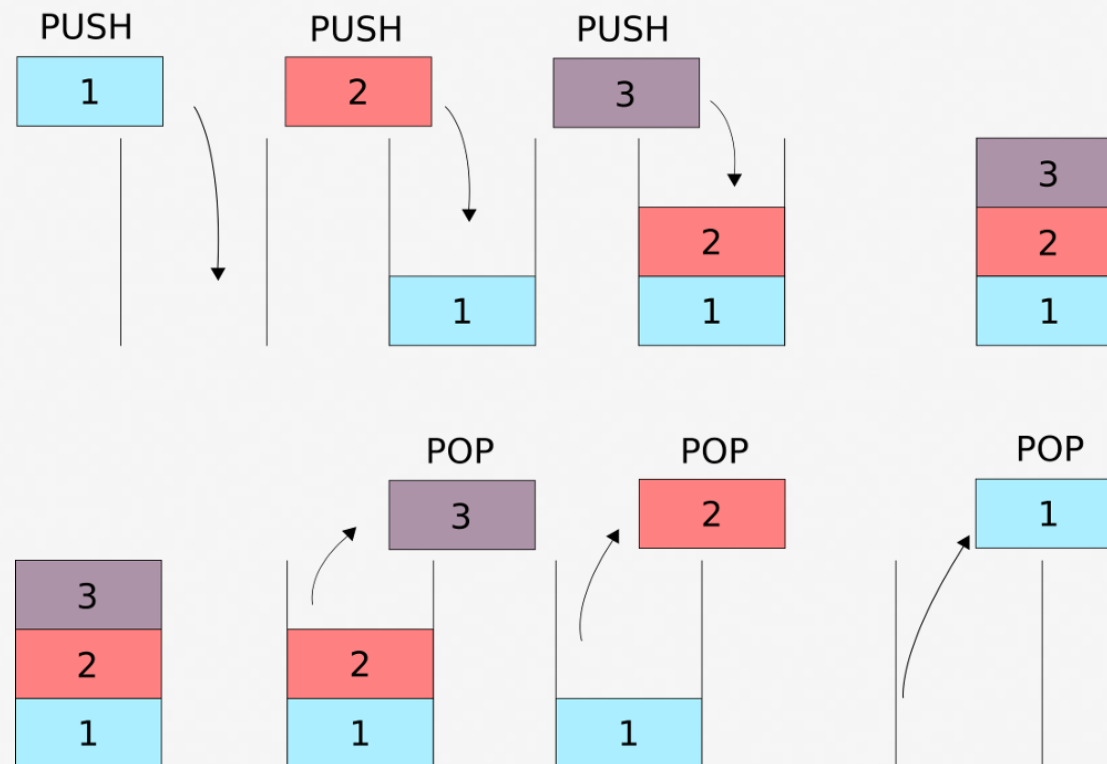
Es un tipo de estructura utilizada para *almacenar* datos, y acceder a ellos en un *orden* específico.

Las pilas son una *colección* ordenada de elementos a los que se puede acceder estrictamente desde la “*cima*”, lo que quiere decir que solo se añaden o se eliminan de la parte superior de la misma.



PILAS (stack)

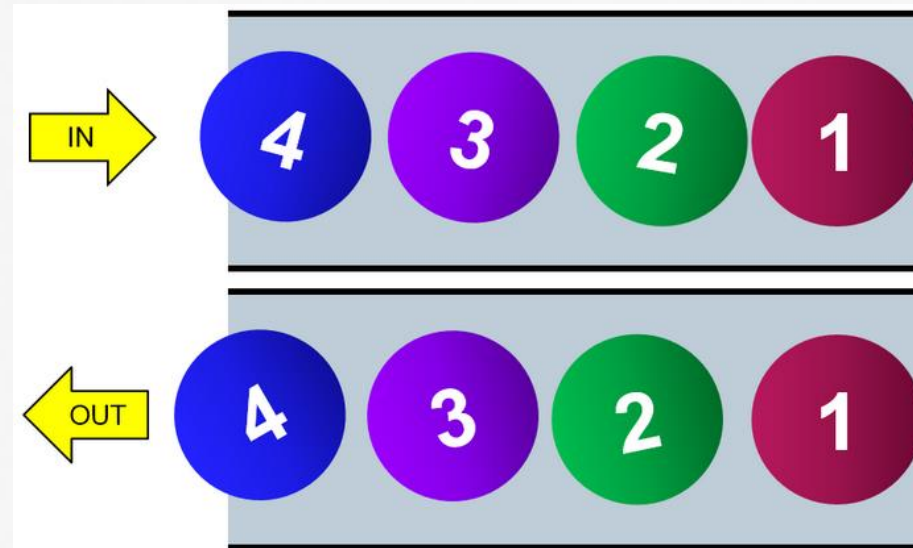
- Es una estructura de datos en donde el último elemento en entrar es el primero en salir (LIFO: Last In – First Out)
- Cada que apilamos un nuevo valor, este pasa a ser la cima de la pila
- Los elementos de la pila se eliminan en estricto orden comenzando por la ‘cima’



PILAS (stack)

En Python Podemos definir pilas bajo las siguientes características:

- Pilas con límite de tamaño
- Pilas de tamaño dinámico (sin límite de tamaño)

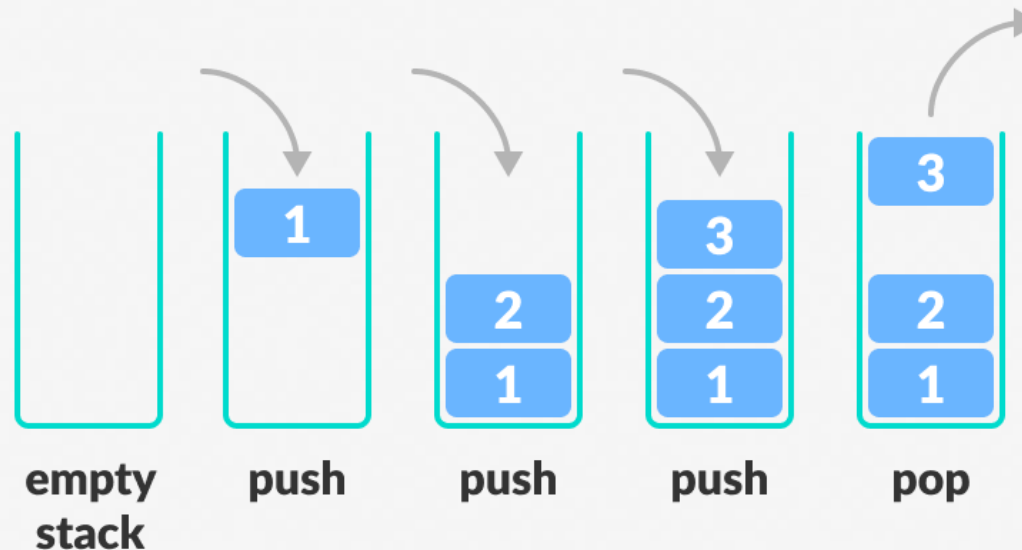


PILAS (stack)

En las pilas las *dos únicas* operaciones posibles son *insertar* y *eliminar*, conocidas también como *push* y *pop*.

Una pila se puede trabajar como un *arreglo*.

Otra forma de trabajar con una pila, es mediante la creación de una *lista enlazada*.

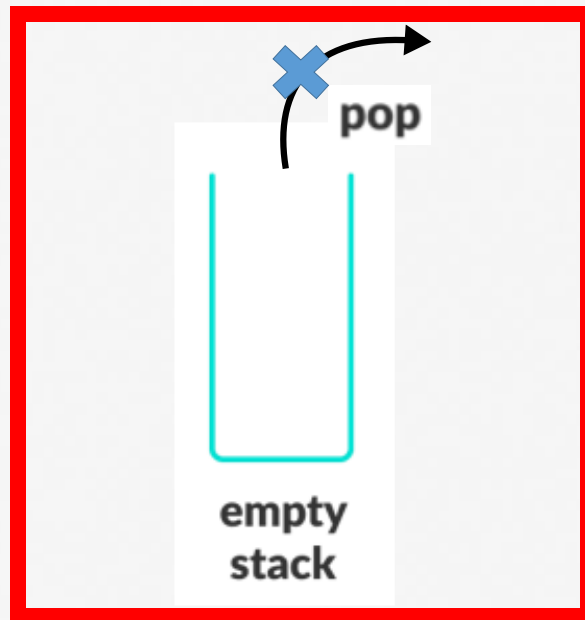


PILAS (stack)

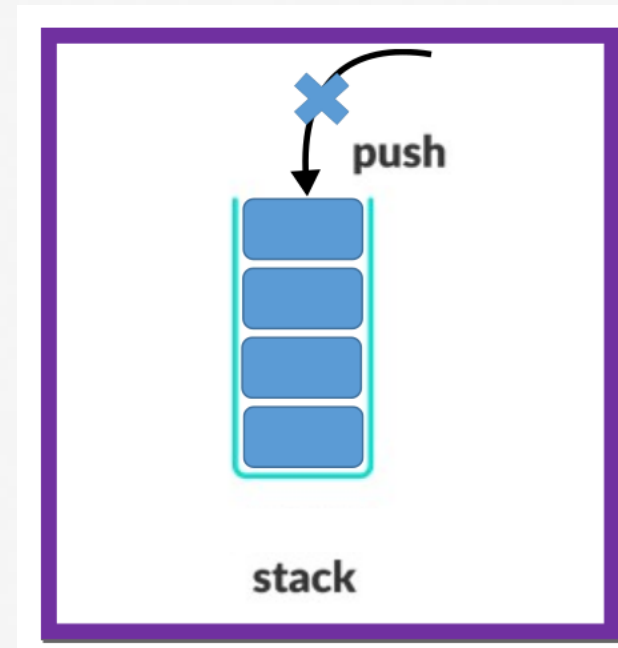
En el caso en que la lista está vacía, es imposible realizar una operación pop (*eliminar*), esta situación se conoce como desbordamiento negativo (*underflow*).

Por el contrario, si se intenta ingresar (*push*) un elemento de una lista llena se producirá un error por desbordamiento de memoria conocido como *overflow*, esto solo es válido cuando hablamos de listas con tamaño fijo.

Underflow



Overflow



PILAS (stack)

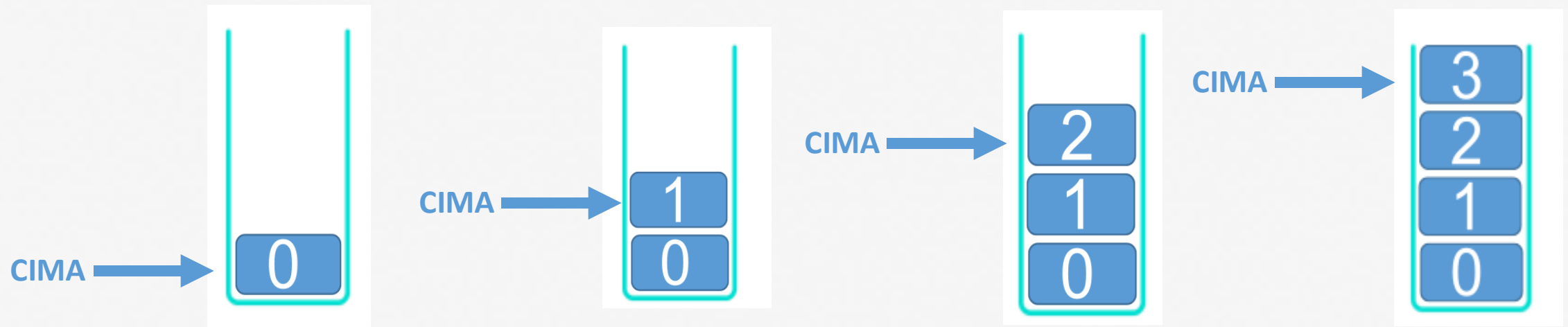
Las operaciones que sirven para la creación y manipulación de las pilas son las siguientes :

<i>CrearPila</i>	Inicia la pila.
<i>Insertar (push)</i>	Pone un dato en la pila.
<i>Quitar (pop)</i>	Retira (saca) un dato de la pila.
<i>Pilavacia</i>	Comprobar si la pila no tiene elementos.
<i>Pilallena</i>	Comprobar si la pila está llena de elementos.
<i>Limpiar pila</i>	Quita todos sus elementos y dejar la pila vacía.
<i>CimaPila</i>	Obtiene el elemento cima de la pila.
<i>Tamaño de la pila</i>	Número de elementos máximo que puede contener la pila.

PILAS A PARTIR DE ARREGLOS

La forma más común de implementar las pilas por medio de arreglos consiste en establecer el fondo de la fila como la **posición 0** del arreglo.

De esta forma cada que se ingrese un dato nuevo se puede ubicar en la posición siguiente, es decir, en la cima de la pila.



PILAS A PARTIR DE ARREGLOS

Definimos la clase stack (pila) que tiene como atributos: la inicialización de un arreglo vacío, el tamaño máximo que puede tener la pila, y el tamaño actual de la pila

```
class Stack:
    def __init__(self, max_size=None):
        self.items = []
        self.max_size = max_size
        self.size = 0
```

El tamaño actual inicia en 0 indicando que la pila está vacía

PILAS A PARTIR DE ARREGLOS

Definimos la clase stack (pila) que tiene como atributos: la inicialización de un arreglo vacío, el tamaño máximo que puede tener la pila, y el tamaño actual de la pila

```
class Stack:  
    def __init__(self, max_size=None):  
        self.items = []  
        self.max_size = max_size  
        self.size = 0
```

```
mi_pila = Stack(3)
```

PILAS A PARTIR DE ARREGLOS

Definimos el método `is_empty` que usamos para saber cuándo una pila está vacía

```
class Stack:
    def __init__(self, max_size=None):
        self.items = []
        self.max_size = max_size
        self.size = 0

    def is_empty(self):
        return len(self.items) == 0
```

Retorna True si la pila
está vacía, caso
contrario retorna False

PILAS A PARTIR DE ARREGLOS

Definimos el método `is_full` que usamos para saber cuándo una pila está llena

```
class Stack:
    def __init__(self, max_size=None):
        self.items = []
        self.max_size = max_size
        self.size = 0

    def is_empty(self):
        return len(self.items) == 0

    def is_full(self):
        return self.max_size is not None and self.size >= self.max_size
```



Preguntamos si el tamaño máximo no es None y si el tamaño actual es mayor o igual al tamaño permitido

PILAS A PARTIR DE ARREGLOS

Definimos el método push que usamos para añadir elementos a la pila (esto también se conoce como apilar)

```
def push(self, item):  
    if self.is_full():  
        raise OverflowError("La pila está llena (overflow)")  
    self.items.insert(0, item)  
    self.size += 1
```

Insertamos el elemento en la posición 0 (cima) de la pila
Y aumentamos el atributo del tamaño actual

La palabra reservada raise la usamos para crear un mensaje tipo error, ese error es de la categoría OverflowError

PILAS A PARTIR DE ARREGLOS

Definimos el método push que usamos para añadir elementos a la pila

```
def push(self, item):  
    if self.is_full():  
        raise OverflowError("La pila está llena (overflow)")  
    self.items.insert(0, item)  
    self.size += 1
```

```
mi_pila.push(1)  
mi_pila.push(2)  
mi_pila.push(3)
```

PILAS A PARTIR DE ARREGLOS

Definimos el método `print_stack` que usamos para imprimir todos los elementos de la pila

```
def print_stack(self):  
    print(self.items)
```

PILAS A PARTIR DE ARREGLOS

Definimos el método `print_stack` que usamos para imprimir todos los elementos de la pila

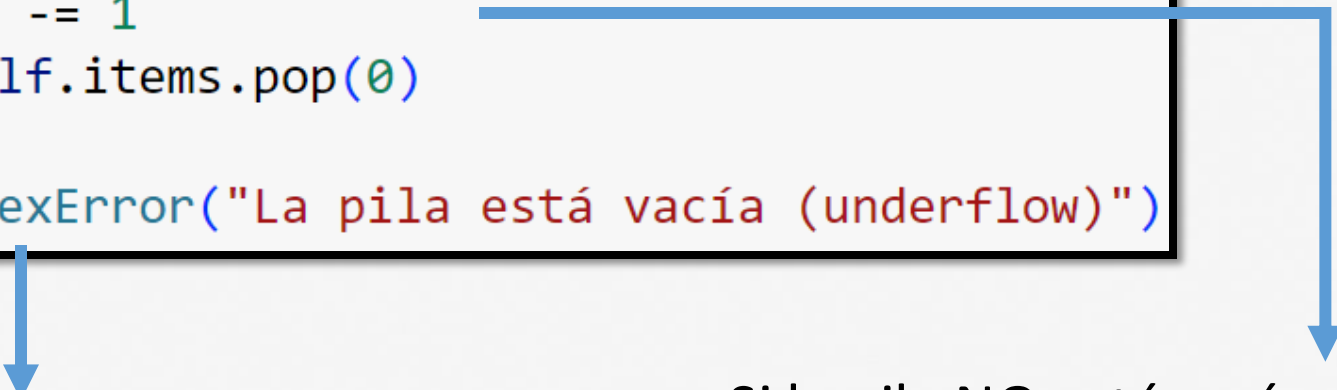
```
def print_stack(self):  
    print(self.items)
```

```
mi_pila.push(1)  
mi_pila.push(2)  
mi_pila.push(3)  
mi_pila.print_stack()
```

PILAS A PARTIR DE ARREGLOS

Definimos el método pop que usamos para eliminar el elemento de la cima en la pila (esto también se conoce como desapilar)

```
def pop(self):  
    if not self.is_empty():  
        self.size -= 1  
        return self.items.pop(0)  
    else:  
        raise IndexError("La pila está vacía (underflow)")
```



La palabra reservada raise la usamos para crear un mensaje tipo error, ese error es de la categoría IndexError

Si la pila NO está vacía, eliminamos el elemento de la posición 0 (cima)
Y disminuimos el tamaño actual de la pila

PILAS A PARTIR DE ARREGLOS

Definimos el método pop que usamos para eliminar el elemento de la cima en la pila (esto también se conoce como desapilar)

```
def pop(self):  
    if not self.is_empty():  
        self.size -= 1  
        return self.items.pop(0)  
    else:  
        raise IndexError("La pila está vacía (underflow)")
```

```
elemento_desapilado = mi_pila.pop()  
print("Elemento desapilado:", elemento_desapilado)  
mi_pila.print_stack()
```


PILAS A PARTIR DE ARREGLOS

Definimos el método `vaciar_pila` que usamos para eliminar todos los elementos de la pila

```
def vaciar_pila(self):  
    self.items = []  
    self.size = 0
```

PILAS A PARTIR DE ARREGLOS

Definimos el método `vaciar_pila` que usamos para eliminar todos los elementos de la pila

```
def vaciar_pila(self):  
    self.items = []  
    self.size = 0
```

```
mi_pila.vaciar_pila()  
mi_pila.print_stack()
```

PILAS A PARTIR DE ARREGLOS

Definimos el método cima que usamos para imprimir el elemento de la cima en la pila

```
def cima(self):  
    if not self.is_empty():  
        return self.items[0]  
    else:  
        raise IndexError("La pila está vacía (underflow)")
```

PILAS A PARTIR DE ARREGLOS

Definimos el método cima que usamos para imprimir el elemento de la cima en la pila

```
def cima(self):  
    if not self.is_empty():  
        return self.items[0]  
    else:  
        raise IndexError("La pila está vacía (underflow)")
```

```
elemento_en_cima = mi_pila.cima()  
print("Elemento en la cima:", elemento_en_cima)
```



Implemente una pila de tamaño dinámico (sin límite de tamaño)

Puede utilizar los códigos ya creados y modificarlos según sea necesario, comience eliminando el atributo `max_size`

En este caso, los errores por overflow no deben tomarse en cuenta



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a curved line on the right side, all in white.